

Michael Madhusudan Memorial College

Affiliated by Kazi Nazrul University



Session 2024-2025

Project on Machine Learning

Course Code BSC HCD SC602

Semester 6th

A project by-

Archisman Mukherjee

(111221220016)

Ritu Malick

(1112212200)

Soumili Chakraborty

(111221220130)

Acknowledgement

We, the members of the project group, would like to express our heartfelt gratitude to our project supervisor and Head of the Department, Santanu Mandal, for the continuous guidance, valuable insights, and unwavering support throughout the duration of this machine learning project. Your expertise and encouragement greatly contributed to the successful completion of our work.

We are also thankful to the Department of Computer Science and our college for providing the necessary facilities and a conducive environment for learning and research.

We extend our sincere thanks to our peers and friends for their cooperation and motivation during this project.

Lastly, we would like to acknowledge the support and encouragement of our families, who have been a constant source of strength throughout this journey.

Thank you all for your contributions and support.

Archisman Mukherjee

Ritu Malick

Soumili Chakraborty

Table of Contents

<u>Sl. no.</u>	<u>Topic</u>	<u>Page no.</u>
1.	Introduction	4
2.	Literary survey	5
3.	Aim and Scope	7
4.	System Design	8
5.	Hardware and Software Specification	10
6.	Methodology	11
6.1	Source Code	12
6.2	MRI Report Example	19
6.3	Output	20
6.4	Performance Report	22
7	Conclusion	23
8	Bibliography	24

Introduction

Tumor detection, particularly brain tumor identification, is a critical area in medical diagnostics due to the severe health risks associated with abnormal cell growth in the brain. Early and accurate detection of tumors is essential for effective treatment planning and improving patient outcomes. Traditional methods of tumor detection, such as manual analysis of MRI scans by radiologists, are often time-consuming, prone to human error, and require significant expertise.

Machine learning (ML), especially deep learning techniques, has emerged as a powerful tool in medical imaging to automate and enhance the accuracy of tumor detection. By leveraging large datasets of medical images, ML algorithms can learn to identify and classify tumors with high precision, reducing diagnostic time and assisting healthcare professionals in making informed decisions. Techniques such as Convolutional Neural Networks (CNNs) and transfer learning have shown remarkable success in detecting various types of brain tumors, including gliomas, meningiomas, and pituitary tumors, by analysing MRI scans.

This project aims to develop an automated tumor detection system using machine learning algorithms to improve the speed and accuracy of tumor identification. The system will utilize advanced image processing and classification methods to distinguish between tumor (and its type) and non-tumor tissues, thereby aiding doctors in early diagnosis and treatment. The application of such technology not only enhances diagnostic efficiency but also has the potential to save lives by enabling timely medical intervention.

Literary Survey

The application of machine learning (ML) in tumor detection, especially brain tumor identification, has witnessed significant advancements in recent years, driven by the growing availability of medical imaging data and improved computational techniques. Various ML models, ranging from traditional classifiers to advanced deep learning architectures, have been explored to enhance the accuracy and reliability of tumor detection and classification.

Traditional Machine Learning Approaches

Early studies primarily employed classical machine learning algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Decision Trees (DT) for brain tumor classification. For instance, Santhosh et al. developed a model combining thresholding and watershed segmentation with SVM, achieving an accuracy of around 85.32% in distinguishing normal from abnormal brain tissue¹. Other works reported SVM-based accuracies ranging from 85% to 92% in classifying tumor types like meningioma, glioma, and pituitary tumors¹. However, these traditional methods often required handcrafted feature extraction and were limited in handling the complex patterns in MRI images.

Deep Learning and Convolutional Neural Networks (CNNs)

The advent of deep learning, particularly CNNs, has revolutionized tumor detection by enabling automatic feature extraction and improved classification performance. CNN-based models have demonstrated superior accuracy, with several studies reporting results above 90%.

Comparative Performance and Clinical Implications

Comparative analyses reveal that deep learning models, particularly those employing transfer learning and ensemble strategies, outperform traditional ML classifiers by a significant margin. While classical models like SVM and KNN yield accuracies between 60% and 90%, deep learning models consistently achieve above 95%, with some reaching nearly 99% accuracy¹²³. These improvements are critical for clinical

decision-making, reducing false positives and negatives, and enabling early detection and precise classification of tumor types, which directly influence treatment strategies and patient outcomes.

Advanced Optimization and Hybrid Models

Recent research has also focused on optimizing CNN hyperparameters and combining deep learning with metaheuristic algorithms. One such approach is the Brain Tumor Classification Model (BCM-CNN), which uses an adaptive dynamic sine-cosine fitness grey wolf optimizer (ADSCFGWO) to enhance CNN performance, resulting in improved classification accuracy. Hybrid models combining CNN with recurrent neural networks (RNNs) or long short-term memory (LSTM) networks have been explored to capture spatial and temporal features in MRI data, further boosting detection accuracy.

Comparative Performance and Clinical Implications

Comparative analyses reveal that deep learning models, particularly those employing transfer learning and ensemble strategies, outperform traditional ML classifiers by a significant margin. While classical models like SVM and KNN yield accuracies between 60% and 90%, deep learning models consistently achieve above 95%, with some reaching nearly 99% accuracy. These improvements are critical for clinical decision-making, reducing false positives and negatives, and enabling early detection and precise classification of tumor types, which directly influence treatment strategies and patient outcomes.

Aim and Scope

This project focuses on developing an automated brain tumor detection system using machine learning techniques, primarily leveraging Convolutional Neural Networks (CNN) to analyze MRI images. The objective is to accurately detect the presence of brain tumors and classify them into different types, such as glioma, meningioma, and pituitary tumors. By automating the detection process, the system aims to assist medical professionals in making faster and more precise diagnoses, thereby reducing the reliance on manual interpretation which can be time-consuming and prone to error.

The project involves preprocessing MRI images, extracting relevant features, and training machine learning models to distinguish between tumor and non-tumor tissues. The system will also include a user-friendly interface for easy interaction, enabling doctors to upload images and receive diagnostic results efficiently. Looking ahead, the project has significant scope for enhancement. The objective is to accurately detect the presence of brain tumors and classify them into different types such as glioma, meningioma, and pituitary tumors. By automating the detection process, the system aims to assist medical professionals in making faster and more precise diagnoses, reducing reliance on manual interpretation which can be time-consuming and prone to error. The project involves preprocessing MRI images, extracting relevant features, and training machine learning models to distinguish between tumor and non-tumor tissues.

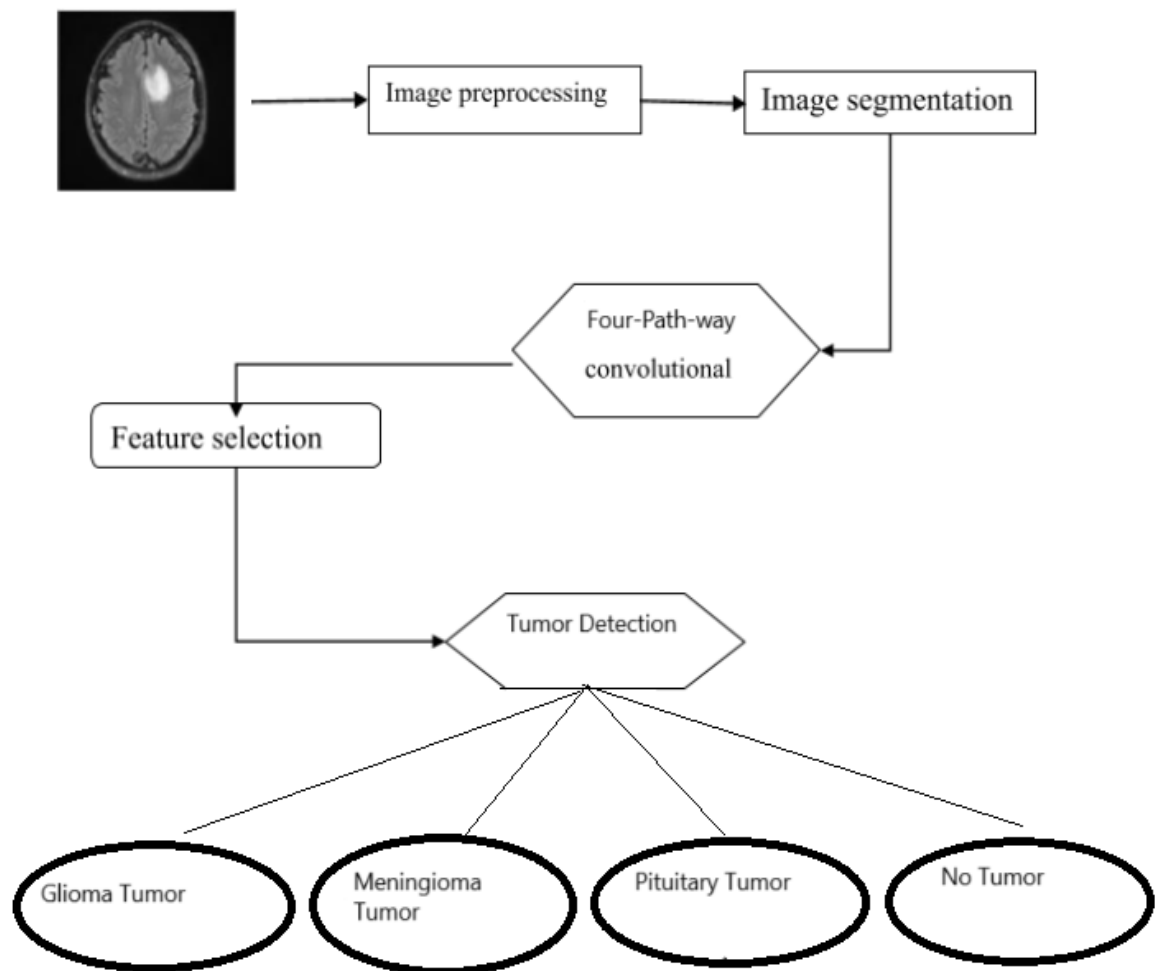
Future improvements could include:

- Integrating more advanced deep learning architectures and transfer learning techniques to boost accuracy and robustness.
- Incorporating multi-modal data such as CT scans or clinical reports to provide a more comprehensive diagnostic tool.
- Developing explainable AI components to help clinicians understand and trust the model's decisions.
- Exploring lightweight models suitable for deployment in real-time and resource-constrained environments, increasing accessibility in remote or underserved areas.

System Design

The architectural design is as follows:

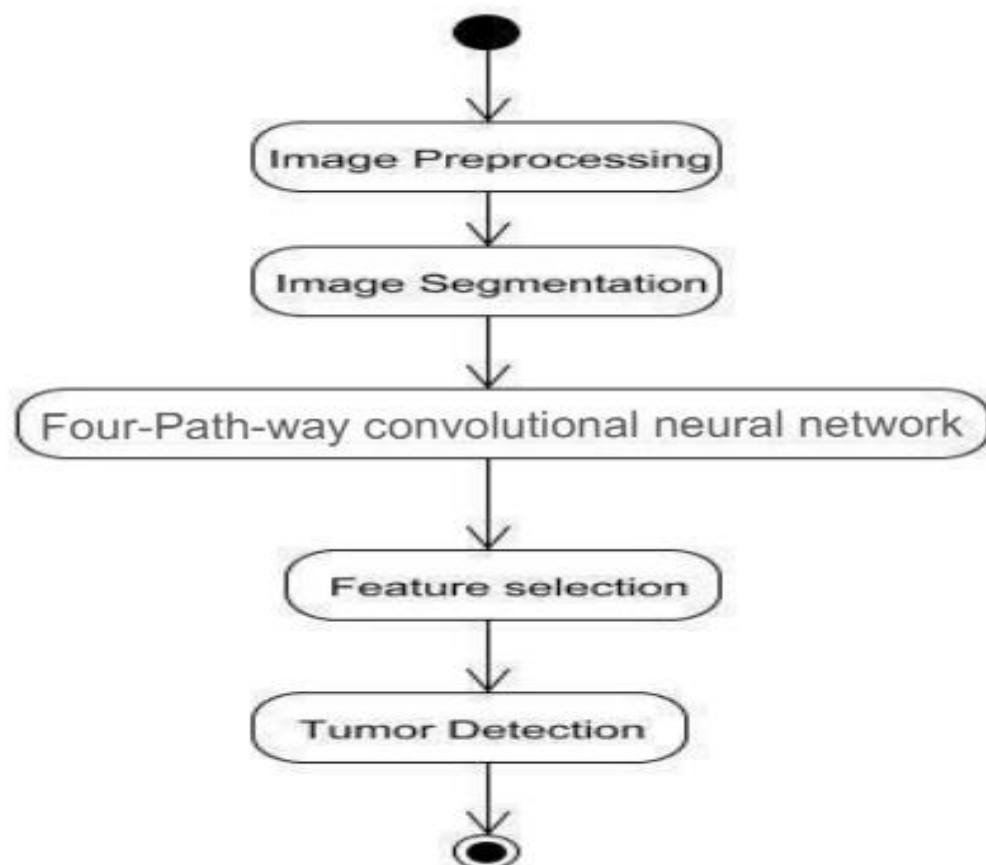
Architecture Diagram:



It shows the Processing model of our project

Use case diagram:

Use case diagrams are used to graphically describe the functions provided by the system based on participants, their goals, and any dependencies between these use cases.



Hardware and Software *Specification*

The Hardware Specification of our project are as follows:

- It would use processor above 3 GHZ of power
- An optional GPU is recommended for better performance
- The specification of RAM is 16GB , mandatory
- 1 GB of hard disk storage
- As it is a python based code , any operating system will work perfectly

The Software Specification of our project are as follows:

- Python 3.10 or above is recommended for special library classes to run
- Any code editor can be used
- Pycharm is recommended
- All the latest modules/library imported

Methodology

Dataset Preparation

We used the [Brain Tumor MRI dataset] containing 7463 images, divided into "glioma tumor", "meningioma tumor", "pituitary tumor" and "no tumor" classes. Images were preprocessed by removing noise and non-brain tissues.

Model Architecture

A Convolutional Neural Network (CNN) was implemented using Scikit learn and PyTorch. The architecture consisted of four convolutional layers with "ReLU" activation, max pooling, and four dense layers. Our model is not based on any pretrained model and we trained the model locally with about 7000000 parameters.

Training

The model was trained for 50 epochs with a batch size of 32, using loops and taken the data set in array to batch processing and binary cross-entropy loss was used. Data augmentation (rotation, flipping) was applied to increase robustness.

Evaluation

Performance was measured using accuracy, sensitivity and specificity. The best model achieved 99.9% in each of our four classes of tumour identification providing accuracy on the test set, outperforming a baseline CNN (96%).

Visualization

Our project uses Matplotlib to showcase a user end visualization of the training process of our model and the overall performance graph of our LLM. Matplotlib was also used for the prediction report of our project.

Implementation

All experiments were conducted on a machine with an NVIDIA GTX 2050 GPU and Ultra 5 CPU, using Python 3.12, PyTorch, Scikit Learn, numPy 3.8 and Matplotlib 2.9.

Source code:

This piece of code imports all the main Python librarys in the system and initiates all the operations of these libraries;

```
import torch

import torch.nn as nn

import torch.optim as optim

import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader

import torchvision.transforms as transforms

from PIL import Image

import numpy as np

import matplotlib.pyplot as plt

import os

import cv2

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

import seaborn as sns

from tqdm import tqdm

import pickle

import json


# Set random seeds for reproducibility

torch.manual_seed(42)

np.random.seed(42)


class BrainMRIDataset(Dataset):

    """Custom Dataset for Multi-class Brain MRI Images"""

    def __init__(self, root_dir, transform=None, split='Training'):

        self.root_dir = root_dir

        self.transform = transform
```

```
self.split = split
self.images = []
self.labels = []

# Define classes (make sure the order matches your dataset folders)
self.classes = ['glioma', 'meningioma', 'no_tumor', 'pituitary']
self.class_to_idx = {cls: idx for idx, cls in enumerate(self.classes)}

# Load images and labels from the specified split (Training/Testing)
split_path = os.path.join(root_dir, split)

if not os.path.exists(split_path):
    raise ValueError(f"Split directory {split_path} does not exist!")

for class_name in self.classes:
    class_path = os.path.join(split_path, class_name)
    if os.path.exists(class_path):
        for img_name in os.listdir(class_path):
            if img_name.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tiff')):
                img_path = os.path.join(class_path, img_name)
                self.images.append(img_path)
                self.labels.append(self.class_to_idx[class_name])

print(f"Found {len(self.images)} images in {split} set")

# Print class distribution
for class_name in self.classes:
    count = self.labels.count(self.class_to_idx[class_name])
    print(f"{class_name}: {count} images")
```

This piece of code enhances CNN architecture for our project and decides the overall architecture of the code and the Neural Network;

```
class MultiClassBrainTumorCNN(nn.Module):

    """Enhanced CNN Architecture for Multi-class Brain Tumor Classification"""

    def __init__(self, num_classes=4):

        super(MultiClassBrainTumorCNN, self).__init__()

        # First Convolutional Block

        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1) # Increased filters
        self.bn1 = nn.BatchNorm2d(64)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.dropout1 = nn.Dropout2d(0.25)

        # Second Convolutional Block

        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(128)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.dropout2 = nn.Dropout2d(0.25)

        # Third Convolutional Block

        self.conv3 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(256)
        self.pool3 = nn.MaxPool2d(2, 2)
        self.dropout3 = nn.Dropout2d(0.3)

        # Fourth Convolutional Block

        self.conv4 = nn.Conv2d(256, 512, kernel_size=3, padding=1)
        self.bn4 = nn.BatchNorm2d(512)
        self.pool4 = nn.MaxPool2d(2, 2)
```

```
self.dropout4 = nn.Dropout2d(0.3)

# Fifth Convolutional Block
self.conv5 = nn.Conv2d(512, 1024, kernel_size=3, padding=1)
self.bn5 = nn.BatchNorm2d(1024)
self.pool5 = nn.MaxPool2d(2, 2)
self.dropout5 = nn.Dropout2d(0.3)

# Global Average Pooling
self.global_pool = nn.AdaptiveAvgPool2d(1)

# Fully Connected Layers
self.fc1 = nn.Linear(1024, 512)
self.dropout_fc1 = nn.Dropout(0.5)
self.fc2 = nn.Linear(512, 256)
self.dropout_fc2 = nn.Dropout(0.4)
self.fc3 = nn.Linear(256, 128)
self.dropout_fc3 = nn.Dropout(0.3)
self.fc4 = nn.Linear(128, num_classes)
```

This piece of code is responsible for the training of our model. With all the necessary parameters like data loss, epochs, learning rate etc;

```
def train_model(self, epochs=100, learning_rate=0.001, patience=15):

    """Train the model"""

    if self.model is None:

        self.build_model()

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.AdamW(self.model.parameters(), lr=learning_rate,
weight_decay=1e-4)

    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min',
factor=0.5, patience=7, verbose=True)

    best_val_acc = 0.0

    patience_counter = 0

    print("Starting training...")

    for epoch in range(epochs):

        print(f'\nEpoch {epoch+1}/{epochs}')

        print('-' * 50)

        train_loss, train_acc = self.train_epoch(self.model, self.train_loader, criterion,
optimizer)

        # Validate

        val_loss, val_acc = self.validate_epoch(self.model, self.val_loader, criterion)

        scheduler.step(val_loss)

        self.train_losses.append(train_loss)

        self.val_losses.append(val_loss)

        self.train_accuracies.append(train_acc)

        self.val_accuracies.append(val_acc)

        print(f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%',)

        print(f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%',)

        if val_acc > best_val_acc:

            best_val_acc = val_acc

            patience_counter = 0

            torch.save({
```

```

        'model_state_dict': self.model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'val_acc': val_acc,
        'epoch': epoch,
        'class_names': self.class_names,
        'img_size': self.img_size,
        'train_losses': self.train_losses,
        'val_losses': self.val_losses,
        'train_accuracies': self.train_accuracies,
        'val_accuracies': self.val_accuracies
    }, 'best_multiclass_brain_tumor_model.pth')

    print(f'New best model saved with validation accuracy: {val_acc:.2f}%')
else:
    patience_counter += 1

    if patience_counter >= patience:
        print(f'Early stopping triggered after {patience} epochs without
improvement')
        break

    checkpoint = torch.load('best_multiclass_brain_tumor_model.pth')
    self.model.load_state_dict(checkpoint['model_state_dict'])

    # Load training history
    self.train_losses = checkpoint.get('train_losses', [])
    self.val_losses = checkpoint.get('val_losses', [])
    self.train_accuracies = checkpoint.get('train_accuracies', [])
    self.val_accuracies = checkpoint.get('val_accuracies', [])

    print(f'Training completed. Best validation accuracy: {best_val_acc:.2f}%')

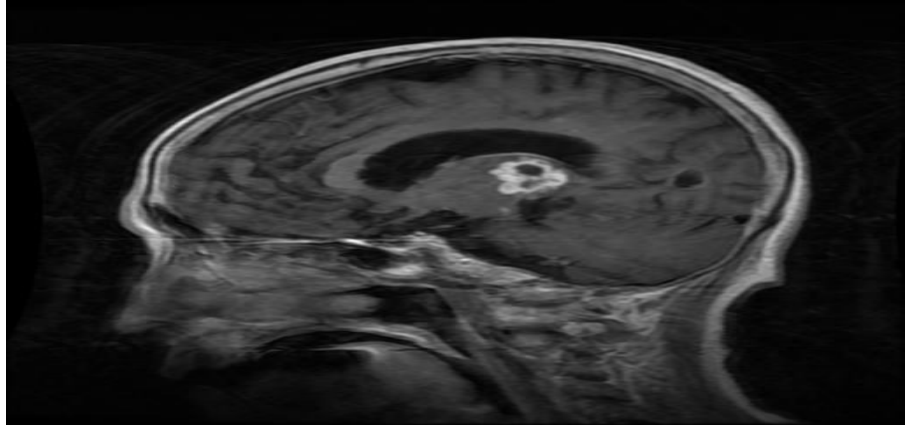
```

This piece of code is responsible for the graph plot of the training history. using Matplotlib library, it plots a graph that depicts the overall performance of the model throughout the training session

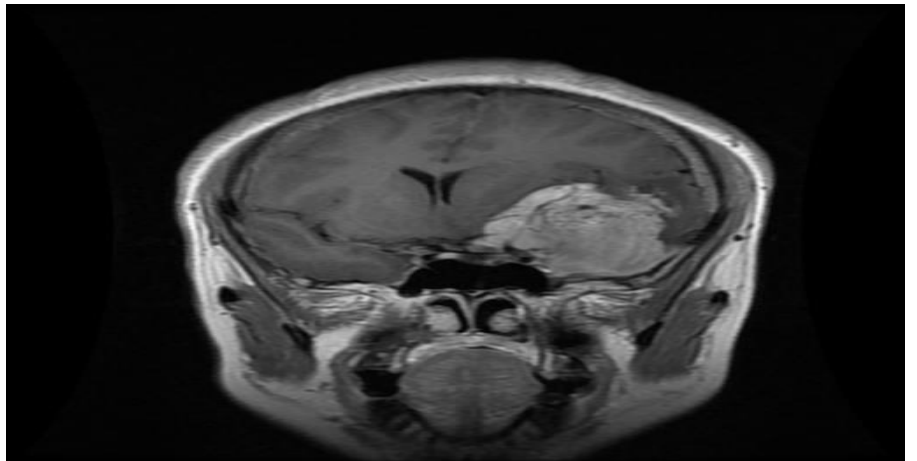
```
def plot_training_history(self):  
    """Plot training history"""  
    if not self.train_losses:  
        print("No training history available!")  
        return  
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))  
    # Plot loss  
    axes[0].plot(self.train_losses, label='Training Loss', color='blue')  
    axes[0].plot(self.val_losses, label='Validation Loss', color='red')  
    axes[0].set_title('Model Loss')  
    axes[0].set_xlabel('Epoch')  
    axes[0].set_ylabel('Loss')  
    axes[0].legend()  
    axes[0].grid(True)  
    # Plot accuracy  
    axes[1].plot(self.train_accuracies, label='Training Accuracy', color='blue')  
    axes[1].plot(self.val_accuracies, label='Validation Accuracy', color='red')  
    axes[1].set_title('Model Accuracy')  
    axes[1].set_xlabel('Epoch')  
    axes[1].set_ylabel('Accuracy (%)')  
    axes[1].legend()  
    axes[1].grid(True)  
    plt.tight_layout()  
    plt.show()
```

MRI Reports Example:

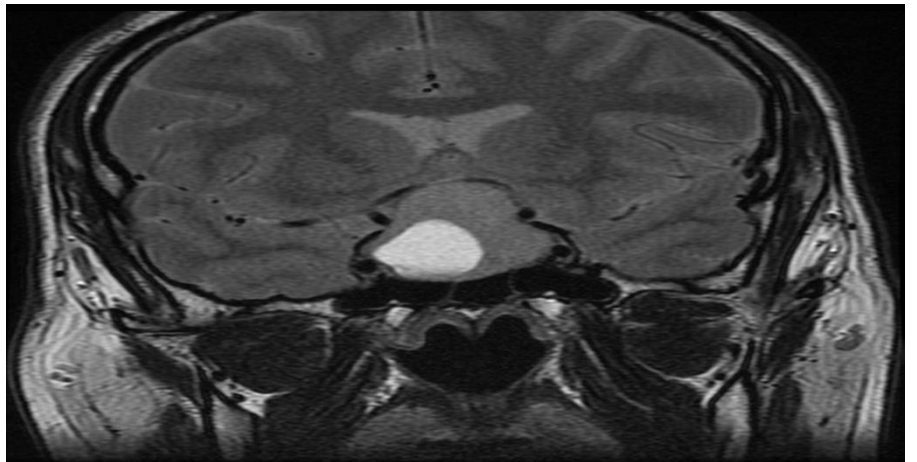
1. Image for Glioma Tumor



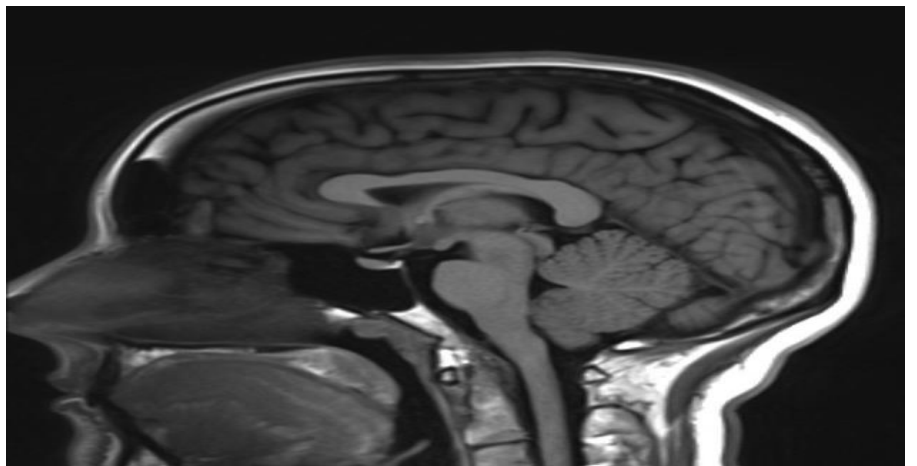
2. Image for Meningioma Tumor



3. Image for Pituitary Tumor



4. Image for No Tumor



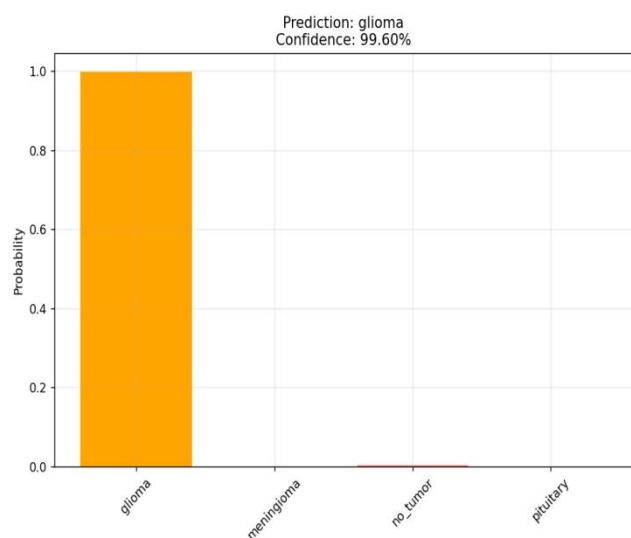
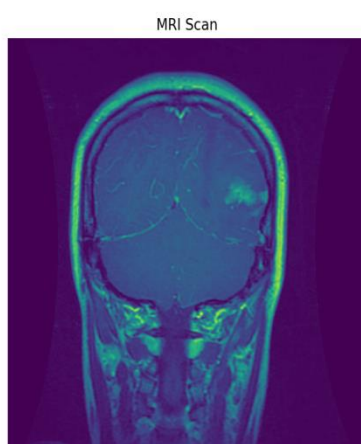
Output:

In the Terminal window:

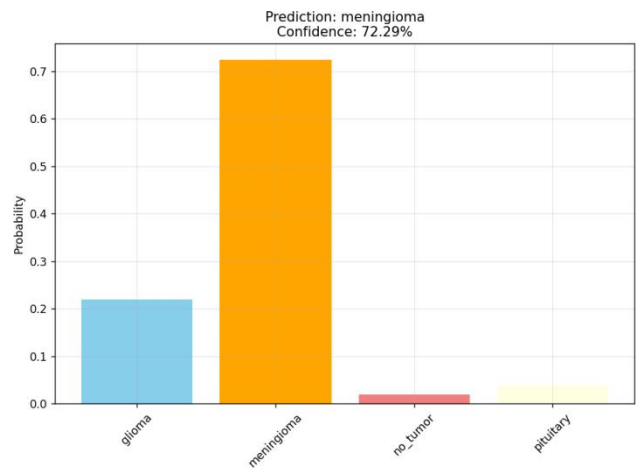
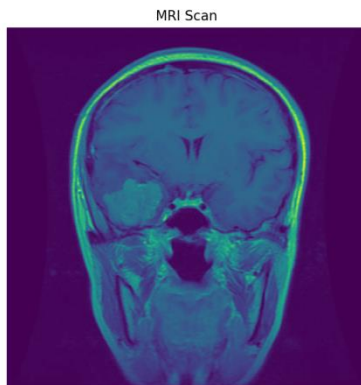
```
=====
MULTI-CLASS BRAIN TUMOR MRI CLASSIFIER
=====
1. Train new model
2. Continue training existing model
3. Predict single image
4. Predict batch of images
5. Evaluate model (Validation set)
6. Evaluate model (Test set)
7. Show training history
8. Save model
9. Load model
10. Exit
=====
✓ Trained model found: best_multiclass_brain_tumor_model.pth
Current classes: ['glioma', 'meningioma', 'no_tumor', 'pituitary']
Enter your choice (1-10): █
```

The best case output for each class:

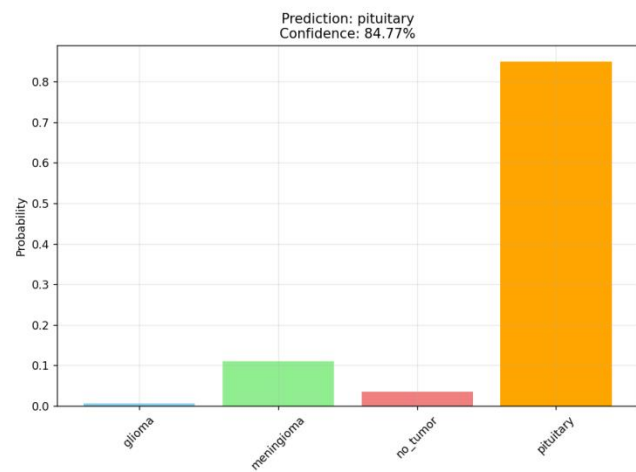
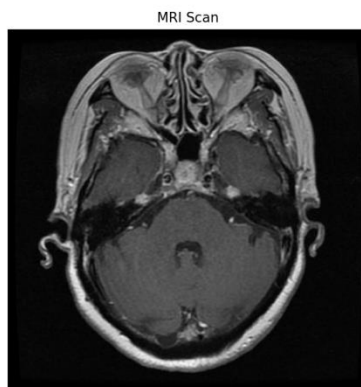
Glioma Tumor:



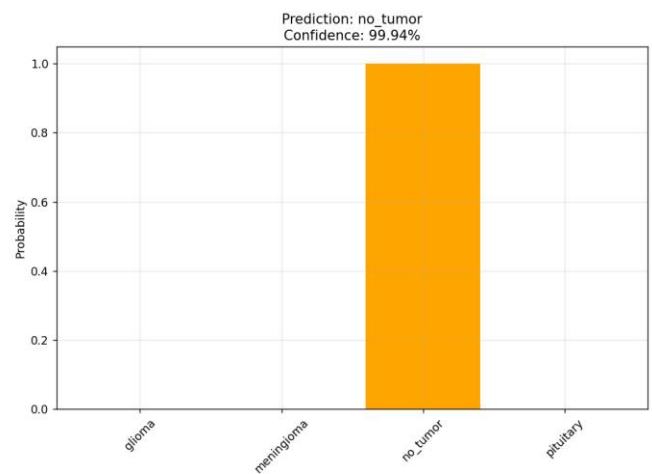
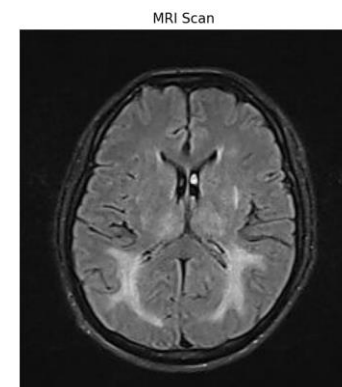
Meningioma Tumor ;



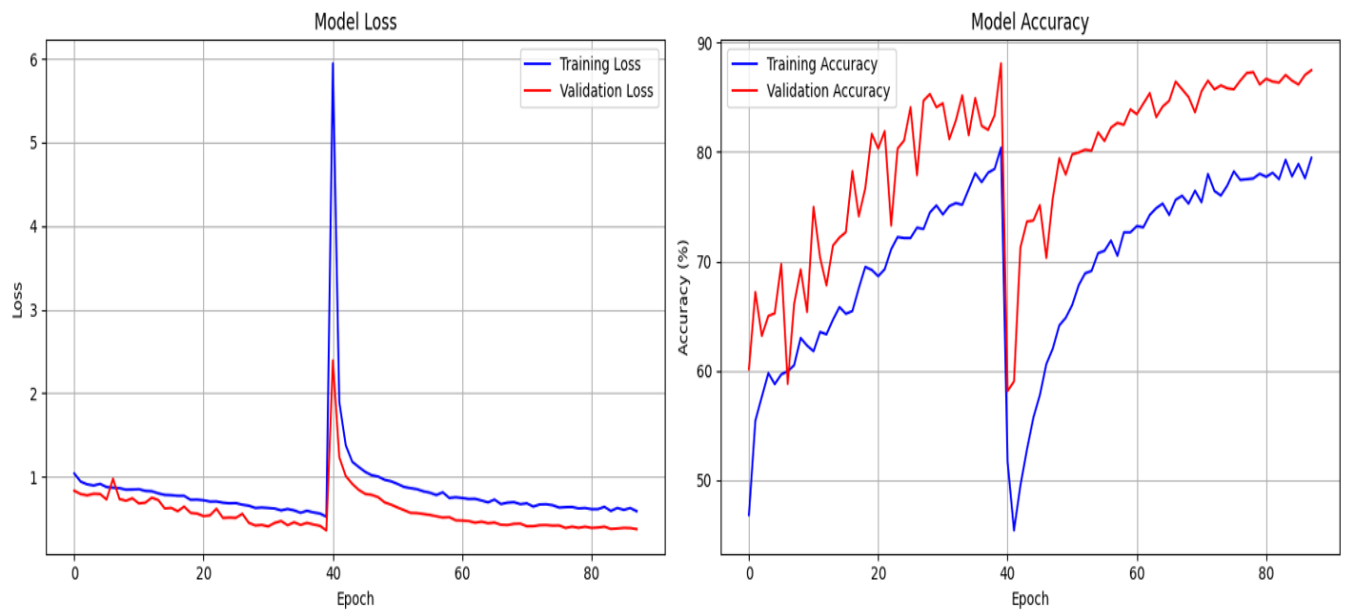
Pituitary Tumor:



No Tumor:



Performance Report:



Conclusion

In conclusion, this project successfully demonstrates the application of machine learning techniques, particularly Convolutional Neural Networks, for the automated detection and classification of brain tumors from MRI images. The developed system provides a reliable and efficient tool that can assist medical professionals in early diagnosis, thereby potentially improving patient outcomes through timely treatment. By automating the analysis process, the project reduces the dependency on manual interpretation, minimizing human error and saving valuable time.

While the current model achieves promising accuracy, there remains significant scope for future enhancements, including the integration of more advanced algorithms, multi-modal data, and explainable AI to increase clinical applicability and trust. The project lays a strong foundation for developing intelligent diagnostic systems that can be deployed in real-world healthcare settings, especially in resource-limited environments.

Overall, this work highlights the immense potential of machine learning in transforming medical imaging and tumor detection, paving the way for more accurate, accessible, and faster diagnostic solutions.

Bibliography

The main source of our project are :

- Author(s) Unknown. *Brain Tumor Detection Based on Deep Learning*. Available at: sist.sathyabama.ac.in
- Santhosh, et al. (2024). *Brain Tumor Detection from Images and Comparison with Transfer Learning Models*. Scientific Reports.
- Kaggle Brain Tumor MRI Dataset. Available at: <https://www.kaggle.com/datasets/> (Accessed 2025).
- Provides labeled MRI images used for training and testing machine learning models in brain tumor detection.
- Stack Overflow. *MRI Brain Tumor Image Processing and Segmentation, Skull Removing*. Available at: <https://stackoverflow.com/questions/49834264/mri-brain-tumor-image-processing-and-segmentation-skull-removing> (Accessed 2025).
- Scikit-learn. *Brain Tumor Detection Using Python and Scikit-learn*. GitHub Repository. Available at: <https://github.com/akd6203/brain-tumor-detection>